UNIT – I

INTRODUCTION TO MACHINE LEARNING

UNIT 1 INTRODUCTION TO MACHINE LEARNING

Machine LearningApplications - Types of Machine Learning Training, Testing, - Machine Learning process Variance Bias - Training Error Testing Error - Overfitting Under fitting Gradient Descent Accuracy Precision Recall Confusion Matrix

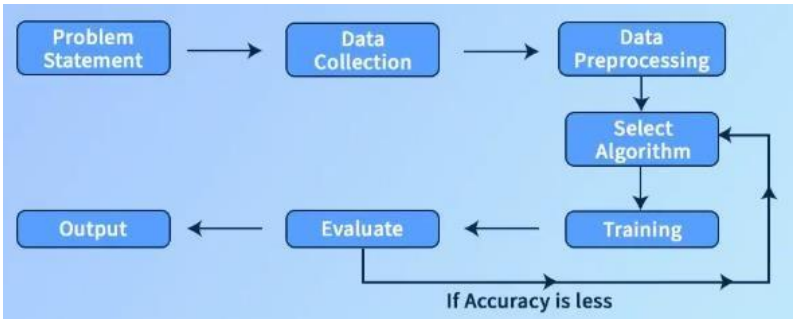
Introduction

Machine Learning is a field of study that gives the computers to Learn Without Being Explicitly Programmed" **"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."**(Tom Michel)

"Field of study that gives computers the ability to learn without being explicitly programmed". Learning = Improving with experience at some task

- Improve over task T,
- with respect to performance measure P,
- based on experience
- .E.g., Learn to lay checkers
- T : Play checkers
- P : % of games won in world tournament
- E: opportunity to play against self

Model



A model of machine learning is a set of programs that can be used to find the pattern and make a decision from an unseen dataset. It can be any one of the following

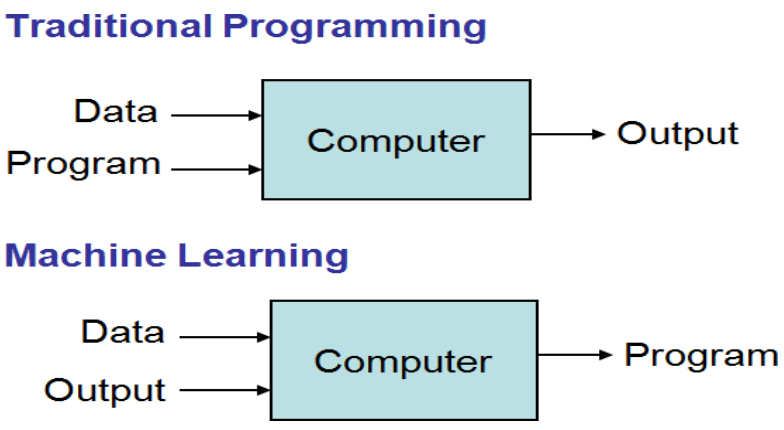
- • Mathematical Equation
- • Relational Diagrams Like Graphs/Trees
- • Logical If/Else Rules
- • Groupings Called Clusters Learning

Training set, Test set and Validation set

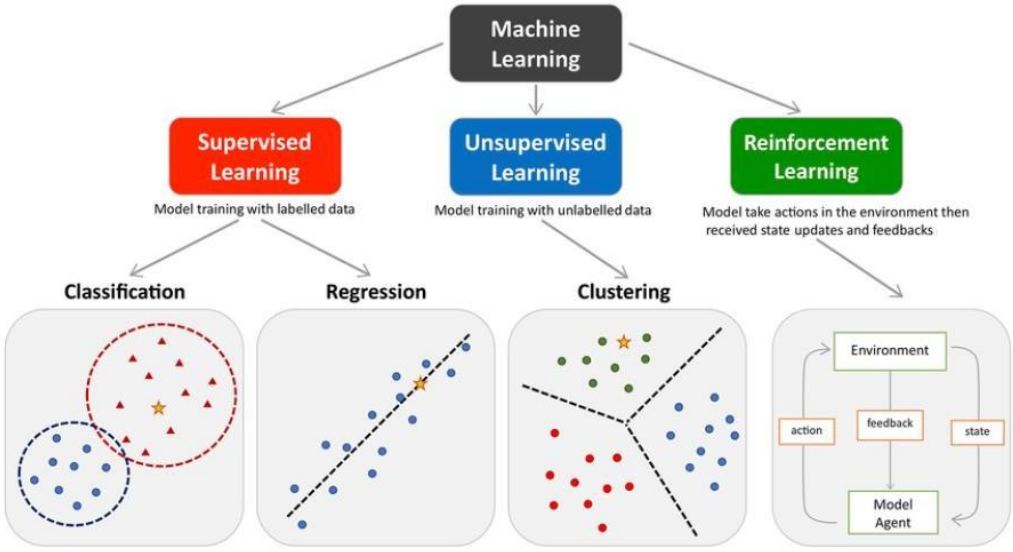
- Divide the total dataset into three subsets:
  - Training data is used for learning the parameters of the model.
  - Validation data is not used of learning but is used for deciding what type of modeland what amount of regularization works best.
  - Test data is used to get a final, unbiased estimate of how well the network works. We expect this estimate to be worse than on the validation data.

We could then re-divide the total dataset to get another unbiased estimate of the true error rate.

DIFFERENCE BETWEEN TRADITIONAL PROGRAMMING VS MACHINE LEARNING



Types of Machine Learning



Supervised learning

Supervised learning is defined as when a model gets trained on a "Labelled Dataset". Labelled datasets have both input and output parameters.

In Supervised Learning algorithms learn to map points between inputs and correct outputs. It has both training and validation datasets labelled.

Example: Consider a scenario where you have to build an image classifier to differentiate between cats and dogs. If you feed the datasets of dogs and cats labelled images to the algorithm, the machine will learn to classify between a dog or a cat from these labeled images. When we input new dog or cat images that it has never seen before, it will use the learned algorithms and predict whether it is a dog or a cat. This is how supervised learning works, and this is particularly an image classification.

There are two main categories of supervised learning that are mentioned below:

Classification- deals with predicting categorical target variables, which represent discre numerical values. For example, predicting the price of a house based on its size, location, and amenities, or forecasting the sales of a product.

Unsupervised learning

Unsupervised learning is a type of machine learning technique in which an algorithm discovers patterns and relationships using unlabeled data.

Unlike supervised learning, unsupervised learning doesn't involve providing the algorithm with labeled target outputs.

The primary goal of Unsupervised learning is often to discover hidden patterns, similarities, or clusters within the data, which can then be used for various purposes, such as data exploration, visualization, dimensionality reduction, and more.

Example: Consider that you have a dataset that contains information about the purchases you made from the shop. Through clustering, the algorithm can group the same purchasing behavior among you and other customers, which reveals potential customers without predefined labels. This type of information can help businesses get target customers as well as identify outliers.

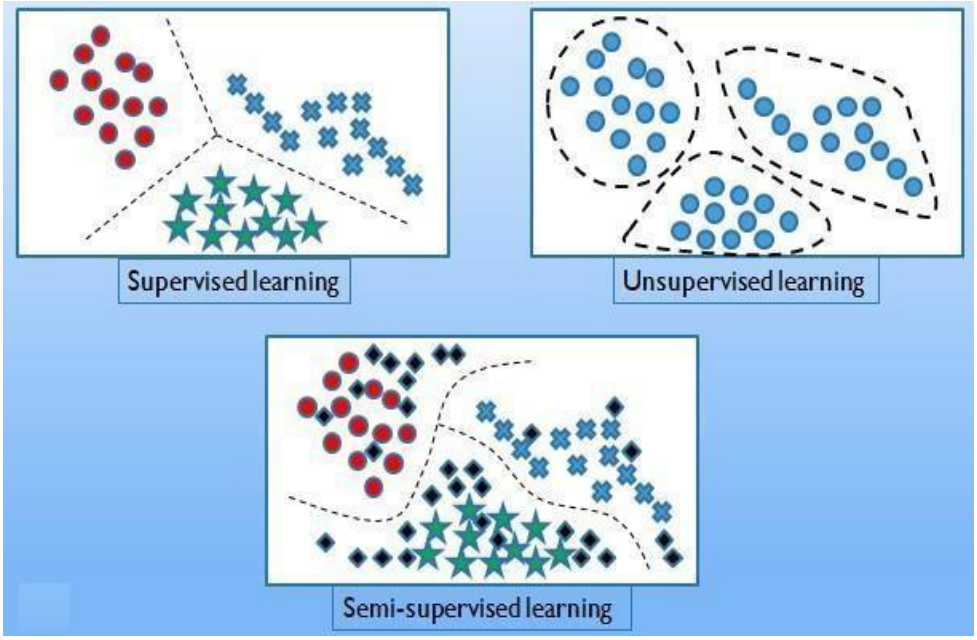
There are two main categories of unsupervised learning that are mentioned below:

Clustering - Clustering is the process of grouping data points into clusters based on their similarity. This technique is useful for identifying patterns and relationships in data without the need for labeled examples.

Association - Association rule learning is a technique for discovering relationships between items in a dataset. It identifies rules that indicate the presence of one item implies the presence of another item with a specific probability.

Semi-supervised learning

- It is the combination of supervised and un supervised learning models.
- Training data includes a few desired outputs



Reinforcement learning

Reinforcement machine learning algorithm is a learning method that interacts with the environment by producing actions and discovering errors.

Trial, error, and delay are the most relevant characteristics of reinforcement learning. In this technique, the model keeps on increasing its performance using Reward Feedback to learn the behavior or pattern.

These algorithms are specific to a particular problem e.g. Google Self Driving car, AlphaGo where a bot competes with humans and even itself to get better and better performers in Go Game.

Each time we feed in data, they learn and add the data to their knowledge which is training data. So, the more it learns the better it gets trained and hence experienced.

Example: Consider that you are training an AI agent to play a game like chess. The agent explores different moves and receives positive or negative feedback based on the outcome. Reinforcement Learning also finds applications in which they learn to perform tasks by interacting with their surroundings.

Examples of machine learning

Recognizing patterns:

– Facial identities or facial expressions

– Handwritten or spoken words

– Medical images

- **Generating patterns:**
  - Generating images or motion sequences (demo)
- **Recognizing anomalies:**
  - Unusual sequences of credit card transactions
  - Unusual patterns of sensor readings in a nuclear power plant or unusual sound in your car engine.
- **Prediction:**

-Future stock prices or currency exchange rates

- **The web contains a lot of data. Tasks with very big datasets often use machine learning**
  - especially if the data is noisy or non-stationary.
- **Spam filtering, fraud detection:**
- **Recommendation systems:**

– Lots of noisy data.

Applications of machine learning

1. **Image Recognition**

Image recognition is one of the most common applications of machine learning. It is used to identify objects, persons, places, digital images, etc. The popular use case of image recognition and face detection is, Automatic friend tagging suggestion:

Facebook provides us a feature of auto friend tagging suggestion. Whenever we upload a photo with our Facebook friends, then we automatically get a tagging suggestion with name, and the technology behind this is machine learning's face

detection and recognition algorithm.

2. **Speech Recognition**

Speech recognition is a process of converting voice instructions into text, and it is also known as "Speech to text", or "Computer speech recognition." At present, machine learning algorithms are widely used by various applications of speech recognition. Google assistant, Siri, Cortana, and Alexa are using speech recognition technology to follow the voice instructions.

3. **Traffic prediction**

It predicts the traffic conditions such as whether traffic is cleared, slow-moving, or heavily congested with the help of two ways:

Real Time location of the vehicle form Google Map app and sensors Average time has taken on past days at the same time.

Everyone who is using Google Map is helping this app to make it better. It takes information from the user and sends back to its database to improve the performance.

4. **Email Spam and Malware Filtering**

Whenever we receive a new email, it is filtered automatically as important, normal, and spam. We always receive an important mail in our inbox with the important symbol and spam emails in our spam box, and the technology behind this is Machine learning. Below are some spam filters used by Gmail:

1. Content Filter
2. Header filter
3. General blacklists filter
4. Rules-based filters
5. Permission filters

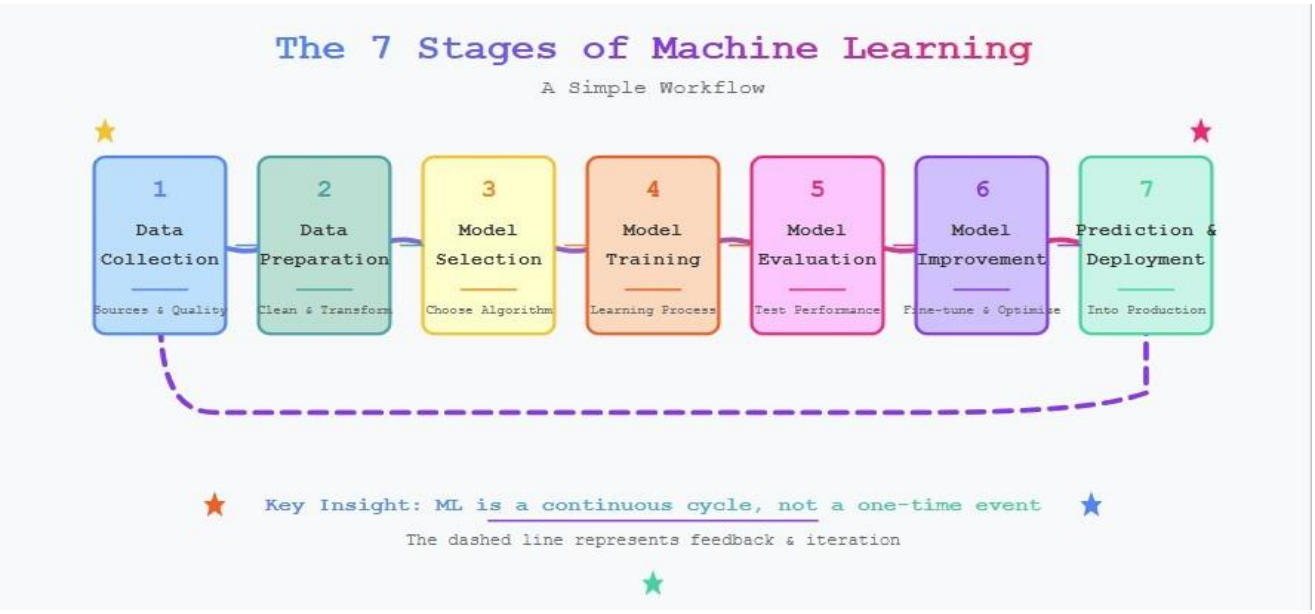
5. **Product recommendations**

Machine learning is widely used by various e-commerce and entertainment companies such as Amazon, Netflix, etc., for product recommendation to the user. Whenever we search for some product on Amazon, then we started getting an advertisement for the same product while internet surfing on the same browser and

this is because of machine learning.

Machine Learning process

Machine Learning Lifecycle is a structured process that defines how machine learning (ML) models are developed, deployed and maintained. It consists of a series of steps that ensure the model is accurate, reliable and scalable.



1. Data Collection: Laying the Foundation

The first and most crucial stage of machine learning is data collection. Just like human learning relies on experience, machine learning depends on data. The quality, volume, and relevance of data significantly influence the model's performance.

Sources of Data:

Databases: Enterprise systems like SAP, Oracle, or CRM platforms

APIs: Public and private APIs (e.g., Twitter API, OpenWeather API)

Web Scraping: Extracting data from websites

IoT Devices and Sensors: In manufacturing, smart homes, or wearables

Manual Entry or Surveys: Useful in early-stage research

2. Data Preparation: Cleaning and Understanding

Raw data is messy. It often contains missing values, duplicates, inconsistencies, and noise. Before you can train a model, data must be cleaned, structured, and transformed. A process that can take up to 80% of the total ML project time.

Key Steps in Data Preparation:

a. Data Cleaning:

Handling Missing Values: Imputation (mean, median, or model-based), or removal

Dealing with Outliers: Using statistical methods (Z-score, IQR) or domain knowledge

Removing Duplicates: Ensures no bias from repeated entries

b. Data Transformation:

Encoding Categorical Variables: One-hot encoding, label encoding

Normalization/Scaling: StandardScaler, MinMaxScaler, RobustScaler for numerical features

Feature Engineering: Creating new variables (e.g., age from date-of-birth)

c. Data Exploration:

Use visualizations like histograms, boxplots, pairplots

Study correlations and feature distributions

Tools: Pandas, NumPy, Matplotlib, Seaborn, Plotly, Tableau

d. Data Augmentation:

In domains like computer vision and NLP, data augmentation techniques improve model generalization. Techniques include image rotation, flipping, scaling, cropping, and even synthetic data generation (e.g., SMOTE for imbalanced classification problems).

These techniques are especially useful when data is scarce or expensive to collect.

3. Choosing the Right Model: Strategy Meets Statistics

Choosing the appropriate model is like selecting the right tool for a job. The model determines how the system interprets data and learns patterns.

Factors That Influence Model Choice:

Type of Problem: Classification, Regression, Clustering, Recommendation, etc.

Size and Structure of Data: Linear vs. nonlinear, small vs. big data

Interpretability Needs: Decision Trees vs. Black-box Neural Networks

Speed and Scalability: Real-time inference or batch processing

Common Algorithms by Problem Type:

Problem Type	Algorithms
Classification	Logistic Regression, Random Forest, SVM, XGBoost, CNN
Regression	Linear Regression, Decision Tree, Lasso/Ridge, Gradient Boosting
Clustering	K-Means, DBSCAN, Hierarchical Clustering
Recommendation	Collaborative Filtering, Matrix Factorization
Time Series	ARIMA, Prophet, LSTM, Exponential Smoothing

4. Training the Model: Teaching the Machine to Learn

Once the model is selected, it's time to feed it data so it can "learn." This is the phase where the machine builds relationships between input features and target outputs.

How It Works:

Data is split into training and validation/test sets (e.g., 80/20)

The algorithm uses the training set to adjust internal parameters (like weights in a neural network)

The aim is to minimize the error or loss function (e.g., MSE, cross-entropy)

Techniques:

Cross-validation: Reduces overfitting by validating on multiple subsets

Early Stopping: Prevents overfitting by halting training when performance stops improving

Regularization: Penalizes complex models (L1, L2) to encourage simplicity

Tools: Scikit-learn, TensorFlow, PyTorch, XGBoost, LightGBM

Outcome: A trained model capable of making predictions on data it has seen during training.

5. Evaluating the Model: How Good Is It, Really?

Training is only half the battle. The real test lies in evaluating how well the model performs on unseen data.

Key Evaluation Metrics:

a. For Classification:

Metric	Use Case
Accuracy	General performance (balanced data)
Precision	Minimizing false positives (e.g., spam detection)
Recall	Minimizing false negatives (e.g., disease detection)
F1 Score	Harmonic mean of precision & recall
ROC-AUC	Measures model's ability to distinguish classes

b. For Regression:

Metric	Use Case
MAE	Mean absolute error
MSE / RMSE	Penalizes larger errors
R^2 / Adjusted R^2	Explains variance captured by the model

c. For Clustering:

Silhouette Score

Davies-Bouldin Index

Visualization Tools:

Confusion Matrix

ROC Curves

Residual Plots

6. Improving the Model: Iterate and Elevate

Very rarely is the first model the best. Model improvement is an iterative process that involves fine-tuning and optimization.

Common Strategies:

a. More Data:

Adds diversity and helps reduce bias

Improves generalization in deep learning models

b. Feature Engineering:

Adding or transforming features can dramatically impact performance

c. Hyperparameter Tuning:
Use Grid Search or Random Search for exhaustive tuning
Modern tools like Optuna and Bayesian Optimization are more efficient

d. Ensemble Learning:
Combines multiple models to improve performance
Techniques: Bagging (Random Forest), Boosting (XGBoost, LightGBM), Stacking

e. Transfer Learning:

Leverage pre-trained models and fine-tune on your data (common in NLP and image tasks)

Tip: Keep a log of all experiments, metrics, and versions for reproducibility.

7. Prediction and Deployment: From Experiment to Production

Once the model performs well on evaluation, it's ready to be deployed. Deployment means integrating the model into a real-world system where it can make predictions on new, unseen data.

Bias and Variance

Bias

Bias is simply defined as the inability of the model because of that there is some difference or error occurring between the model's predicted value and the actual value. These differences between actual or expected values and the predicted values are known as error or bias error or error due to bias. Bias is a systematic error that occurs due to wrong assumptions in the machine learning process.

Let Y be the true value of a parameter, and let $\hat{Y}$ be an estimator of Y based on a sample of data. Then, the bias of the estimator $\hat{Y}$ is given by:

$$\text{Bias}(\hat{Y}) = E(\hat{Y}) - Y$$

Where $E(\hat{Y})$ is the expected value of the estimator $\hat{Y}$. It is the measurement of the model that how well it fits the data.

Low Bias: Low bias value means fewer assumptions are taken to build the target function. In this case, the model will closely match the training dataset.
High Bias: High bias value means more assumptions are taken to build the target function. In this case, the model will not match the training dataset closely.

Variance

Variance is the measure of spread in data from its mean position. In machine learning variance is the amount by which the performance of a predictive model changes when it is trained on different subsets of the training data. More specifically, variance is the variability of the model that how much it is sensitive to another subset of the training dataset. i.e. how much it can adjust on the new subset of the training dataset.

Let Y be the actual values of the target variable, and $\hat{Y}$ be the predicted values of the target variable. Then the variance of a model can be measured as the expected value of the square of the difference between predicted values and the expected value of the predicted values.

$$\text{Variance} = E[(\hat{Y} - E[\hat{Y}])^2]$$

Where $E[\bar{Y}]$ is the expected value of the predicted values. Here expected value is averaged over all the training data**.**
Variance errors are either low or high-variance errors.

Low variance: Low variance means that the model is less sensitive to changes in the training data and can produce consistent estimates of the target function with different subsets of data from the same distribution. This is the case of underfitting when the model fails to generalize on both training and test data.
High variance: High variance means that the model is very sensitive to changes in

the training data and can result in significant changes in the estimate of the target function when trained on different subsets of data from the same distribution. This is the case of overfitting when the model performs well on the training data but poorly on new, unseen test data. It fits the training data too closely that it fails on the new training dataset.

Training error

Training error is the error measured on the **training dataset**, i.e., the data used to train the machine learning model. It shows how well the model has learned the patterns in the training data.

- Calculated **only on training data**
- Indicates how well the model fits the training set
- Lower training error means better fit on training data

Training error is the difference between the **actual output** and the **predicted output** of a model on the **training data**.

Formula

For regression:

$$\text{Training Error} = \frac{1}{n}\sum(y - \hat{y})^2$$

(Mean Squared Error – MSE)

For classification:

$$\text{Training Error} = \frac{\text{Number of incorrect predictions}}{\text{Total training samples}}$$

Advantages

- Helps check whether the model is learning
- Useful for debugging model training

Limitation

- A very low training error does **not guarantee** good performance on new (unseen) data

Testing error

Testing error is the error measured on the test dataset, which consists of unseen data not used during training. It indicates how well a machine learning model generalizes to new data.

- Calculated on **unseen (test) data**
- Measures **model performance and generalization**
- More reliable than training error for real-world prediction

Formula

For regression:

$$\text{Testing Error} = \frac{1}{n}\sum(y - \hat{y})^2$$

(Mean Squared Error – MSE)

For classification:

$$\text{Testing Error} = \frac{\text{Number of incorrect predictions}}{\text{Total test samples}}$$

Training Error vs Testing Error

Aspect	Training Error	Testing Error
Definition	Error calculated on training data	Error calculated on test (unseen) data
Dataset used	Training dataset	Testing dataset
Purpose	Measures how well the model fits training data	Measures how well the model generalizes
Data visibility	Seen by the model during training	Not seen by the model during training
Error value	Usually lower	Usually higher than training error
Indicates	Learning capability of the model	Real-world performance
Used for	Checking model fit	Model evaluation and selection
Overfitting sign	Very low training error	High testing error
Underfitting sign	High training error	High testing error
Reliability	Less reliable for real-world use	More reliable for prediction
Example metrics	MSE, MAE, training accuracy	MSE, MAE, test accuracy
Exam importance	Shows learning performance	Shows generalization ability

Overfitting

Overfitting occurs when a machine learning model learns the **training data too well**, including noise and minor details, and therefore performs **poorly on new (unseen) data**.

Definition:

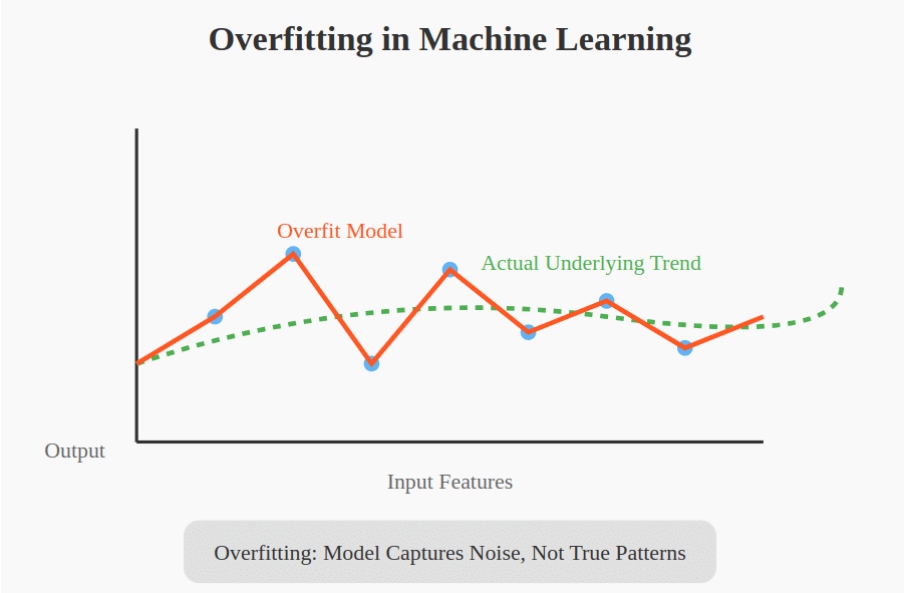
Overfitting is a condition where a model has **very low training error but high testing error**.

Characteristics

- Excellent performance on training data
- Poor performance on test/real-world data
- Model becomes **too complex**

Causes of Overfitting

- Too many features
- Very complex model (deep trees, large neural networks)
- Small training dataset
- Training for too many epochs



Underfitting

Underfitting occurs when a machine learning model is **too simple** to capture the underlying patterns in the data, resulting in **poor performance on both training and testing data**.

Definition:

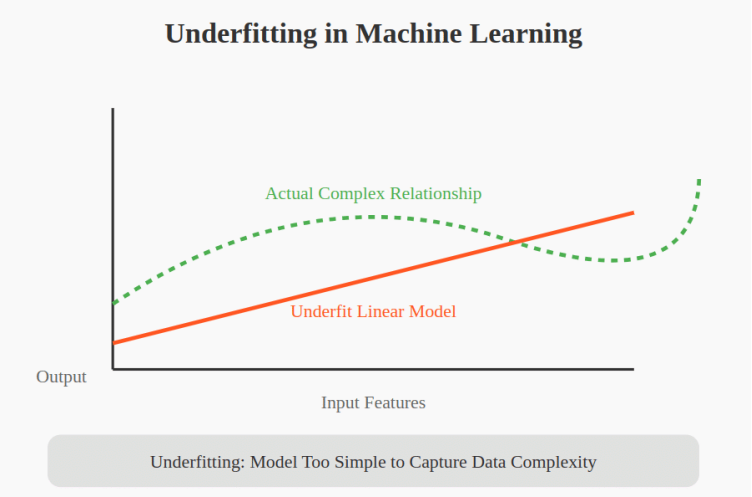
Underfitting is a condition where a model has **high training error and high testing error**.

Characteristics

- Model is overly simple
- Fails to learn important relationships
- Poor accuracy on all datasets

Causes of Underfitting

- Insufficient or irrelevant features
- Very simple model (e.g., linear model for complex data)
- Too few training iterations
- Excessive regularization



Aspect	**Overfitting**	**Underfitting**
Definition	Model learns training data too well, including noise	Model is too simple to learn data patterns
Model complexity	Very high	Very low
Training error	Very low	High
Testing error	High	High
Generalization	Poor	Poor
Cause	Too many features, complex model, small dataset	Insufficient features, simple model, poor training
Performance on new data	Poor	Poor

Aspect	Overfitting	Underfitting
Bias	Low bias	High bias
Variance	High variance	Low variance
Example	Deep decision tree	Linear model for complex data
Solution	Regularization, cross-validation, simplify model	Increase complexity, add features

Gradient Descent

Gradient Descent is a widely used **optimization algorithm** in machine learning and deep learning. It is used to **minimize the cost (loss) function** by iteratively adjusting model parameters in the direction of **steepest descent**.

Gradient Descent is a fundamental optimization algorithm that plays a crucial role in training machine learning models by minimizing error through iterative updates.

Definition

Gradient Descent is an iterative optimization technique that updates model parameters by moving in the **negative direction of the gradient** of the loss function to reach the minimum error.

Objective of Gradient Descent

- To find the **optimal values of parameters**
- To minimize the **loss function**
- To improve model accuracy

Mathematical Equation

$$\theta = \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

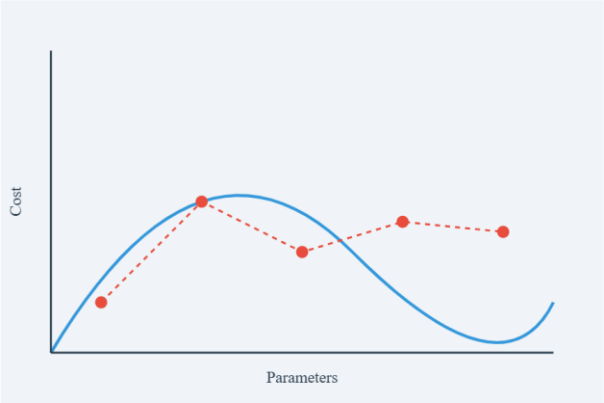
Where:

- θ= model parameter
- α= learning rate
- $J(\theta)$= cost function

Steps in Gradient Descent

1. Initialize parameters with random values
2. Compute the cost function
3. Calculate the gradient of the cost function
4. Update parameters using the learning rate
5. Repeat steps until convergence

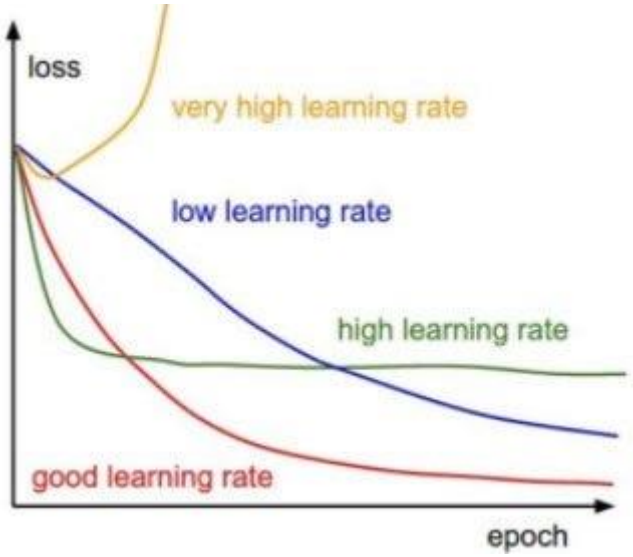
Diagram 1: Gradient Descent on Cost Function



Learning Rate (α)

- Controls the **step size** of parameter updates
- Small learning rate → slow convergence
- Large learning rate → may overshoot minimum

Diagram 2: Effect of Learning Rate



Types of Gradient Descent

Type	Description
Batch Gradient Descent	Uses entire dataset per update
Stochastic Gradient Descent	Uses one data point at a time
Mini-batch Gradient Descent	Uses small batches of data

Advantages

- Simple and easy to implement
- Works well with large datasets
- Efficient for continuous optimization

Limitations

- Can converge to local minima
- Sensitive to learning rate
- Slow for complex cost functions

Applications

- Linear regression
- Logistic regression
- Neural network training

Confusion Matrix

Confusion matrix is a simple table used to measure how well a classification model is performing. It compares the predictions made by the model with the actual results and shows where the model was right or wrong. This helps you understand where the model is making mistakes so you can improve it. It breaks down the predictions into four categories:

True Positive (TP): The model correctly predicted a positive outcome i.e the actual outcome was positive.
True Negative (TN): The model correctly predicted a negative outcome i.e the actual outcome was negative.
False Positive (FP): The model incorrectly predicted a positive outcome i.e the actual

outcome was negative. It is also known as a Type I error.

False Negative (FN): The model incorrectly predicted a negative outcome i.e the actual outcome was positive. It is also known as a Type II error.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Metrics based on Confusion Matrix Data

1. Accuracy

Accuracy shows how many predictions the model got right out of all the predictions. It gives idea of overall performance but it can be misleading when one class is more dominant over the other. For example a model that predicts the majority class correctly most of the time might have high accuracy but still fail to capture important details about other classes. It can be calculated using the below formula:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Measures overall correctness of the model.

2. Precision

Precision focus on the quality of the model's positive predictions. It tells us how many of the "positive" predictions were actually correct. It is important in situations where false positives need to be minimized such as detecting spam emails or fraud. The formula of precision is:

$$Precision = \frac{TP}{TP + FP}$$

Measures how many predicted positives are actually positive.

3. Recall

Recall measures how how good the model is at predicting positives. It shows the proportion of true positives detected out of all the actual positive instances. High recall is essential when missing positive cases has significant consequences like in medical tests.

$$Recall = \frac{TP}{TP + FN}$$

Measures how many actual positives are correctly identified.

4. F1-Score

F1-score combines precision and recall into a single metric to balance their trade-off. It provides a better sense of a model's overall performance particularly for imbalanced datasets. It is helpful when both false positives and false negatives are important though it assumes precision and recall are equally important but in some situations one might matter more than the other.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Problem

A spam classifier produces the following confusion matrix:

Actual \ Predicted	**Spam**	**Not Spam**
Spam	90	10
Not Spam	20	80

Find:
a) Accuracy
b) Precision
c) Recall

Solution

TP = 90, FN = 10, FP = 20, TN = 80

$$Accuracy = \frac{90 + 80}{90 + 80 + 20 + 10} = \frac{170}{200} = 85\%$$

$$Precision = \frac{90}{90 + 20} = 81.81\%$$

$$Recall = \frac{90}{90 + 10} = 90\%$$